

Arquitetura do Conhecimento: Python e R para Analytics

Estruturas de Dados e Lógica

Organização de informações complexas usando listas e dicionários, como um sistema de cadastro

Performance com Pandas e NumPy

Processamento eficiente de tabelas e operações matemáticas vetorizadas para análise de grandes volumes

Dataviz Interativo (Streamlit)

Transformação de scripts em dashboards web dinâmicos com filtros e gráficos de visualização interativa

O Fluxo R: Estatística e Tidyverse

Manipulação fluente com Dplyr e Tidy, organizando dados como uma receita de passos lógicos

O Motor Python: Fundamentos e Manipulação

Fundação Unificada: Google Colab / Jupyter (Ambiente Teórico e Prático Hands-on)

Gramática dos Gráficos (ggplot2)

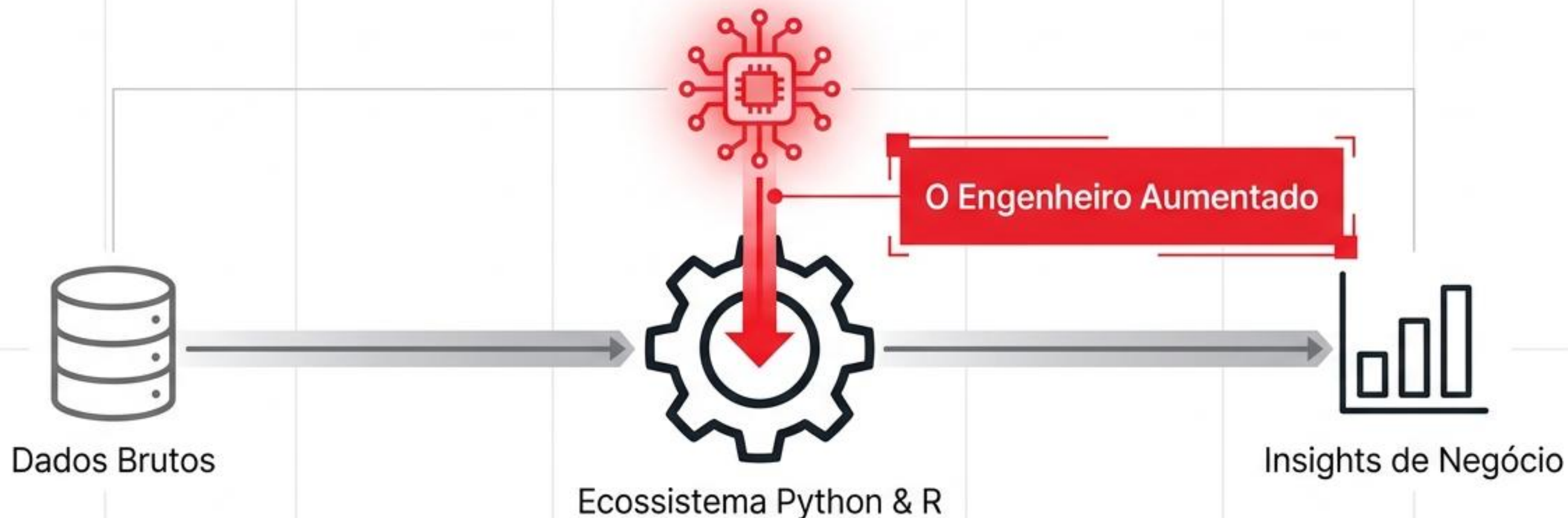
Criação de visualizações profissionais e estatísticas (histogramas, boxplots) através de camadas de dados

Pipelines e Casos Reais

Integração de todo o fluxo analítico para resolução de problemas de negócio e apresentação final

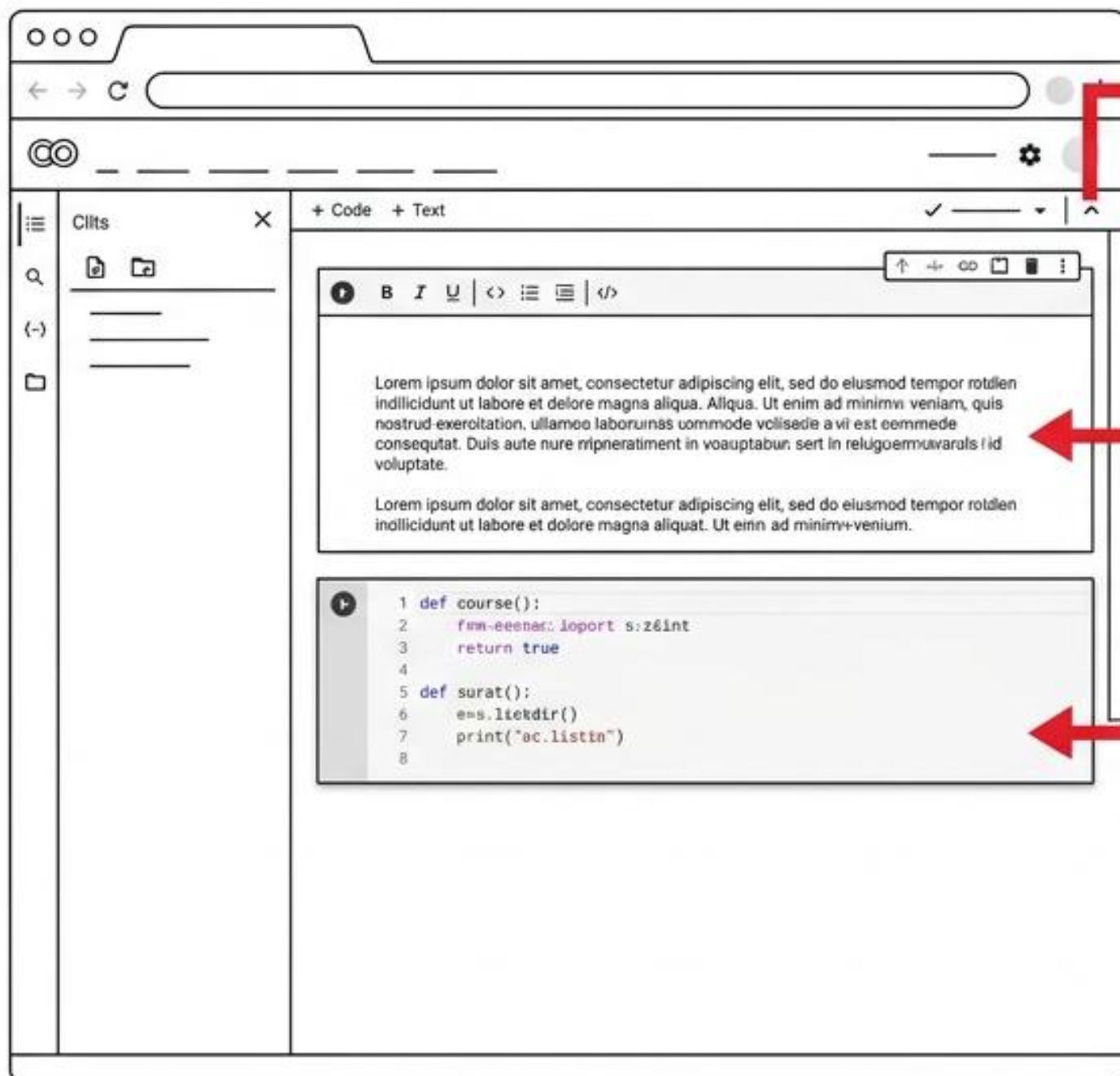
Tecnologia	Módulos	Carga Horária
Python (Básico ao Dash)	1 ao 4	16 h/aula
R (Estatística e Tidy)	5 ao 8	16 h/aula

Python e R para Dados: O Caminho da Engenharia Analítica



O papel do Python transcende a sintaxe. O foco moderno é construir pipelines resilientes e resolver problemas de negócios complexos, com a IA atuando como o copiloto que acelera a execução técnica.

Google Colab: Seu Laboratório na Nuvem



Zero Configuração & Hardware Dedicado
(Acesso a GPUs T4, L4)

Células de Markdown:
Na engenharia de dados, documentar o raciocínio é tão crítico quanto o código.

Células de Código: O motor de execução.

Power Moves

Shift + Enter

Executa a célula atual e avança.

Ctrl + M + M

Transforma instantaneamente em Markdown.

os.listdir()

Integração direta com o sistema de arquivos e Google Drive.

A separação entre Markdown e Código permite que o notebook seja simultaneamente um software executável e um relatório executivo legível.

Escrevendo Código Pythonic

"O Zen of Python (Tim Peters)"

Bonito é melhor que feio.

Explícito é melhor que implícito.

Simples é melhor que complexo.

O Custo da Complexidade

```
resultado = []
for i in dados:
    if i != None:
        if i % 2 == 0:
            resultado.append(i)
```

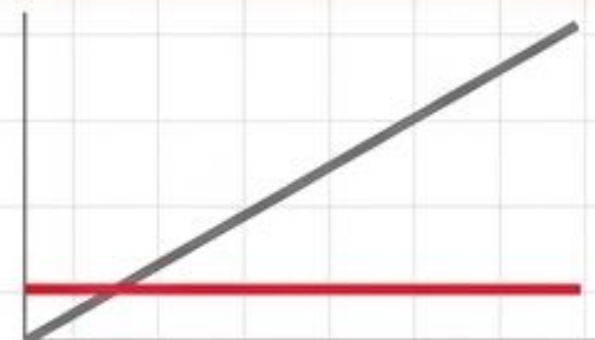
A Elegância Pythonic

```
resultado = [i for i in dados if i and i % 2 == 0]
```

Insight: Máquinas apenas executam o código; humanos o mantêm. Código legível e 'Pythonic' reduz drasticamente os **custos operacionais** e de manutenção técnica a longo prazo.

Estruturas Nativas de Alta Performance

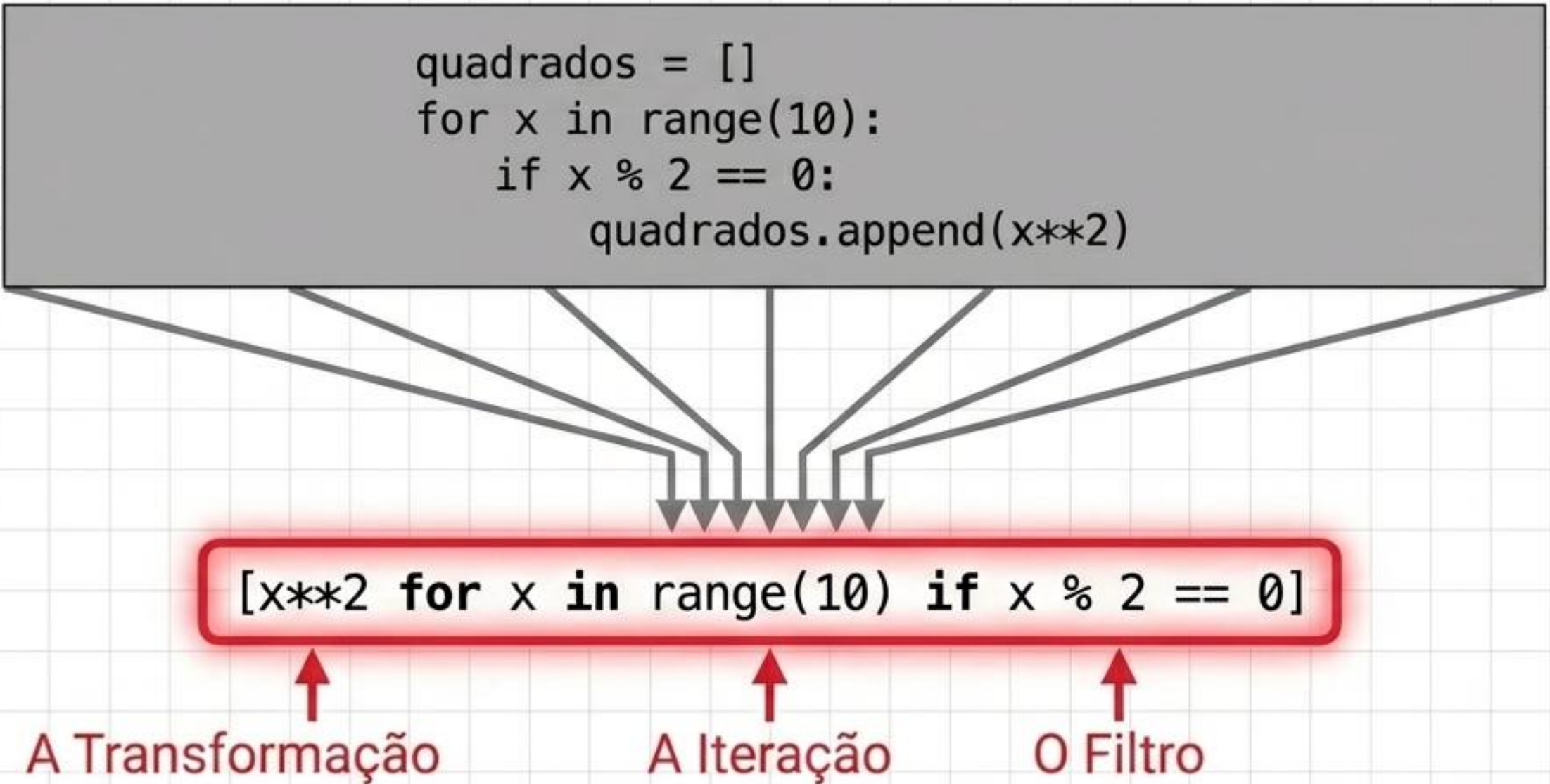
	Listas [...]	Tuplas (...)	Dicionários {...}
Mutabilidade	Mutáveis	Imutáveis	Mutáveis
Casos de Uso na Engenharia	Filas de processamento sequencial	Credenciais, coordenadas imutáveis	Parse de payloads JSON de APIs
Performance / Big-O	Busca Lenta: $O(N)$	Busca Lenta: $O(N)$	Acesso Rápido $O(1)$



Por que importa? Buscar um item em um dicionário $O(1)$ salva horas de processamento em pipelines de Big Data comparado a iterar sobre listas.

Controle de Fluxo e Iteração Avançada

```
quadrados = []  
for x in range(10):  
    if x % 2 == 0:  
        quadrados.append(x**2)
```



```
[x**2 for x in range(10) if x % 2 == 0]
```

A Transformação

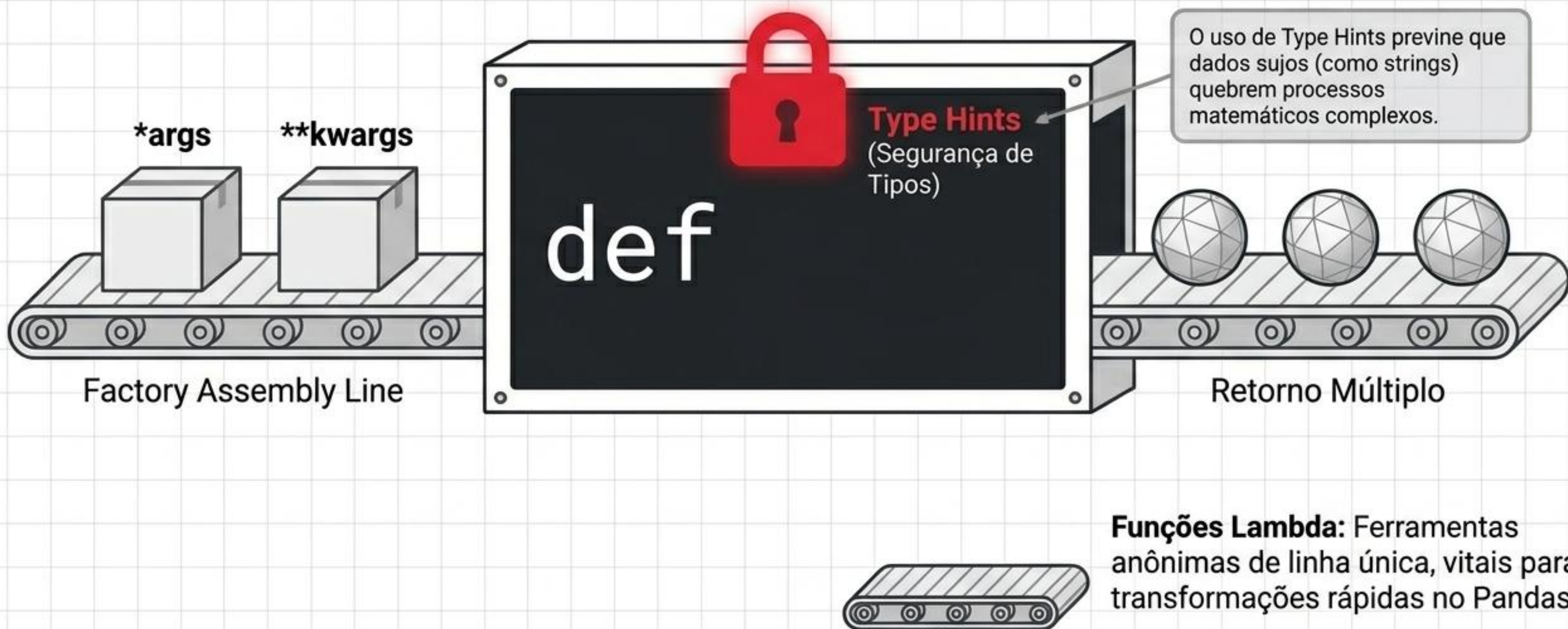
A Iteração

O Filtro

Insight de Negócio: List Comprehensions substituem loops lentos e verbosos. Para volumes moderados de dados, essa sintaxe é otimizada a nível de sistema (C), economizando tempo de execução e entregando uma arquitetura de código limpa.

Criando Funções Reutilizáveis

Na engenharia analítica, nós abandonamos scripts monolíticos em favor de pipelines construídos com blocos funcionais rigorosamente definidos.



Engenharia Aumentada por IA



Casos de Uso Ideais

- ✓ Refatoração de código legado (Converter Loops for em Comprehensions)
- ✓ Debugging reverso de logs de erro complexos
- ✓ Geração e teste de Expressões Regulares (Regex)

O Perigo



Alucinações de Código

LLMs produzem sintaxe plausível, porém logicamente falha ou subotimizada. O rigor na revisão arquitetural humana é inegociável.

Como Pedir Código para a IA de forma Eficiente

[C]

Contexto: O estado atual do sistema.

Ação: Explique o formato exato dos seus dados de entrada (schemas, tipos primitivos, amostras de JSON).

[I]

Instrução: O que deve ser executado.

Ação: Seja imperativo e específico sobre a biblioteca (ex: NumPy) e a metodologia matemática desejada.

[R]

Restrição: Onde o negócio e o hardware importam.

Ação: Imponha limites reais. Ex: 'Otimize para o mínimo consumo de memória RAM' ou 'A complexidade de tempo deve ser $O(1)$ '.

Desafio Prático: Parse de Logs com IA

Fase 1: A Realidade do Negócio

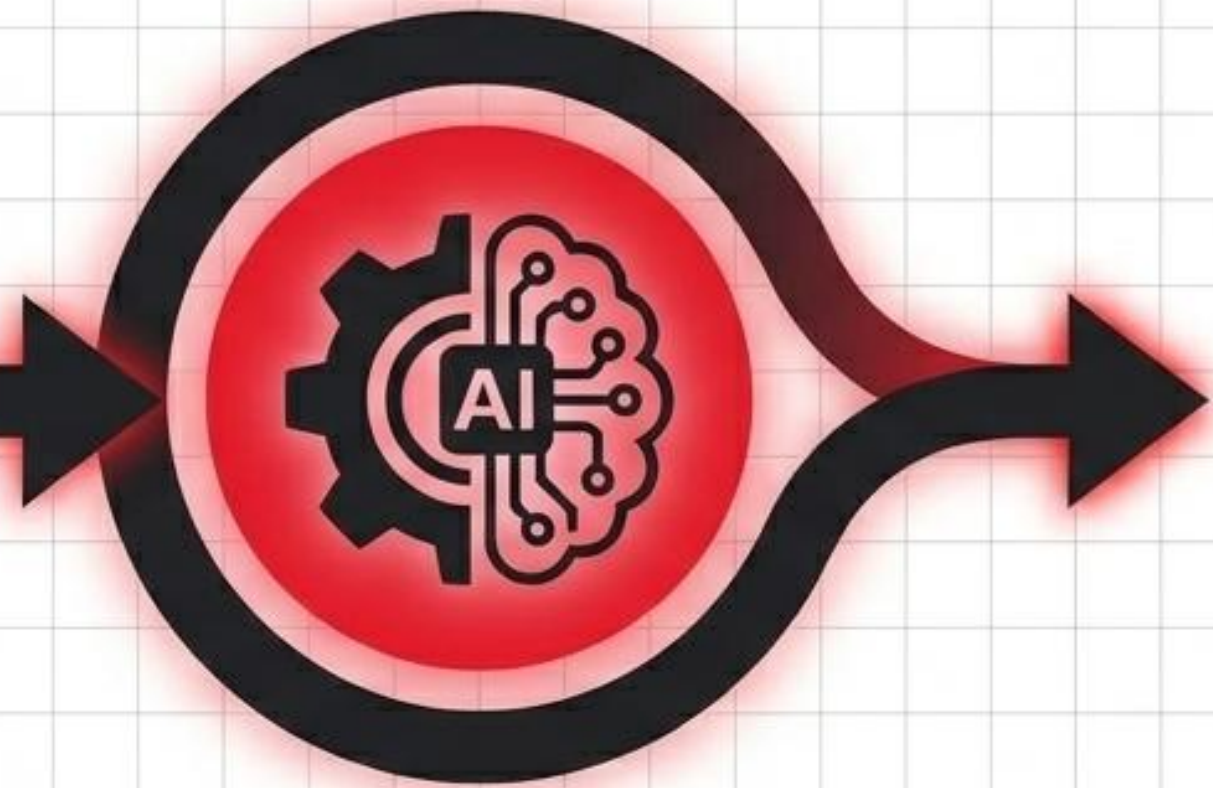
192.168.1
connection
timeout...
2023-10-27T
server_id=1
GET /api/v1/
500 Internal
...

[ERROR]
connection
18

192.168.1.1
2023-10-27T
server_id=5
GET /api/v1/
500 Internal
...

192.168.1...
GET /api/v1/
500 Internal
...

Fase 2: O Motor Híbrido



Fase 3: A Engenharia de Valor

```
{  
  "logs": [  
    {  
      "ip": "192.168.1.100",  
      "timestamp": "2023-10-27T10:15:30Z",  
      "level": "ERROR",  
      "message": "connection timeout",  
      "endpoint": "/api/v1/data",  
      "status_code": 500  
    },  
    {  
      "ip": "10.0.0.5",  
      "timestamp": "2023-10-27T10:15:35Z",  
      "level": "INFO",  
      "message": "user login success",  
      "endpoint": "/login",  
      "status_code": 200  
    }  
  ]  
}
```

Objetivo:	Ferramentas:	Meta:
Limpar um arquivo de texto sujo contendo logs de acesso web não estruturados.	Python puro (dicionário O(1) + IA Generativa para construção de Expressões Regulares (Regex).	Arquivo JSON estruturado, pronto para ingestão em Data Warehouses.

**Abra seu Google Colab.
É hora de construir.**